

Homework 4: Model Evaluation and Comparison

BEE 4850/5850, Fall 2025

Name:

ID:

Due Date

Friday, 3/27/26, 9:00pm

Overview

Instructions

The goal of this homework assignment is to practice model evaluation and using cross-validation and information criteria to distinguish between models.

- Problem 1 asks you to fit a model for cherry blossom bloom dates and use cross-validation to draw conclusions about generalizability.
- Problem 2 asks you to compare several linear regression models for environmental influences on non-accidental death data.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```

Activating project at `~/Documents/Spring_2026/BEE4850/hw/hw04-avinasharavind`
Precompiling project...
1524.8 ms EnumX
1259.1 ms Adapt
1494.1 ms EpollShim_jll
1496.9 ms Libmount_jll
542.7 ms Xorg_libICE_jll
554.0 ms Xorg_libpciaccess_jll
548.8 ms Xorg_libXau_jll
1906.6 ms MbedTLS
595.6 ms LERC_jll
567.6 ms mtdev_jll
565.6 ms Xorg_libXdmcp_jll
704.7 ms JpegTurbo_jll
718.4 ms XZ_jll
775.5 ms Arpack_jll
702.7 ms Xorg_xtrans_jll
710.2 ms libevdev_jll
774.4 ms LZO_jll
816.5 ms Zstd_jll
787.4 ms eudev_jll
751.1 ms Dbus_jll
983.7 ms ColorVectorSpace → SpecialFunctionsExt
2908.1 ms SpecialFunctions → SpecialFunctionsChainRulesCoreExt
2241.4 ms StatsFuns → StatsFunsChainRulesCoreExt
3151.6 ms OpenSSL
2446.8 ms Distributions → DistributionsTestExt
879.3 ms ArrayInterface
733.8 ms OffsetArrays → OffsetArraysAdaptExt
1580.9 ms Distributions → DistributionsChainRulesCoreExt
1749.1 ms Adapt → AdaptSparseArraysExt
789.2 ms Wayland_jll
840.7 ms Xorg_libSM_jll
849.1 ms libdrm_jll
796.8 ms Xorg_libxcb_jll
697.4 ms libinput_jll
1548.7 ms Arpack
2066.7 ms Ghostscript_jll
2292.1 ms Libtiff_jll
1047.2 ms ArrayInterface → ArrayInterfaceChainRulesCoreExt
1376.7 ms ArrayInterface → ArrayInterfaceSparseArraysExt
586.8 ms ArrayInterface → ArrayInterfaceStaticArraysCoreExt
724.3 ms Xorg_xcb_util_jll

```

727.1 ms	Xorg_libX11_jll
1155.6 ms	FiniteDiff
784.5 ms	Xorg_xcb_util_image_jll
11144.4 ms	StaticArrays
2888.3 ms	MultivariateStats
712.5 ms	Xorg_xcb_util_keysyms_jll
650.0 ms	Xorg_xcb_util_wm_jll
668.3 ms	Xorg_xcb_util_renderutil_jll
725.0 ms	Xorg_libXrender_jll
729.2 ms	Xorg_libXext_jll
769.3 ms	Xorg_libXfixes_jll
739.1 ms	Xorg_libxkbfile_jll
1254.7 ms	FiniteDiff → FiniteDiffSparseArraysExt
1370.3 ms	DifferentiationInterface → DifferentiationInterfaceFiniteDiffExt
5461.2 ms	Latexify
1629.8 ms	StaticArrays → StaticArraysChainRulesCoreExt
1069.1 ms	StaticArrays → StaticArraysStatisticsExt
1156.3 ms	FillArrays → FillArraysStaticArraysExt
1084.3 ms	ConstructionBase → ConstructionBaseStaticArraysExt
1063.3 ms	DifferentiationInterface → DifferentiationInterfaceStaticArraysExt
684.0 ms	Xorg_xcb_util_cursor_jll
1052.6 ms	Adapt → AdaptStaticArraysExt
1042.3 ms	FiniteDiff → FiniteDiffStaticArraysExt
692.1 ms	Xorg_libXinerama_jll
691.5 ms	Xorg_libXrandr_jll
680.5 ms	Libglvnd_jll
678.7 ms	Xorg_libXcursor_jll
713.6 ms	libva_jll
716.7 ms	Xorg_libXi_jll
1313.9 ms	Pango_jll
774.9 ms	Xorg_xkbcomp_jll
1692.9 ms	Latexify → SparseArraysExt
1720.1 ms	NLSolversBase
1396.4 ms	Unitful → LatexifyExt
3049.7 ms	Latexify → DataFramesExt
1624.8 ms	Xorg_xkeyboard_config_jll
1778.8 ms	LineSearches
695.5 ms	xkbcommon_jll
6125.7 ms	FFMPEG
6794.2 ms	Interpolations
1131.2 ms	Vulkan_Loader_jll
7751.2 ms	NearestNeighbors
999.4 ms	libdecor_jll

```

2673.1 ms    Interpolations → InterpolationsUnitfulExt
5638.1 ms    Optim
 720.0 ms    GLFW_jll
3576.4 ms    Clustering
6035.1 ms    KernelDensity
27952.6 ms   HTTP
 7435.6 ms   Qt6Base_jll
4308.1 ms   Qt6ShaderTools_jll
4324.4 ms   Qt6Svg_jll
4405.2 ms   GR_jll
2678.2 ms   Qt6Declarative_jll
 394.8 ms   Qt6Wayland_jll
3452.1 ms   GR
54816.6 ms   Plots
4123.0 ms   Plots → UnitfulExt
7015.0 ms   StatsPlots
100 dependencies successfully precompiled in 112 seconds. 199 already precompiled.

```

The following packages are included in the environment (to help you find other similar packages in other languages). The code below loads these packages for use in the subsequent notebook (the desired functionality for each package is commented next to the package).

```

using Random # random number generation and seed-setting
using DataFrames # tabular data structure
using DataFramesMeta # API which can simplify chains of DataFrames transformations
using CSV # reads/writes .csv files
using Distributions # interface to work with probability distributions
using Plots # plotting library
using StatsBase # statistical quantities like mean, median, etc
using StatsPlots # some additional statistical plotting tools
using Optim # optimization package for model fitting
using Dates # API for date-time data structures

```

```

blossoms = CSV.read("data/kyoto.csv", DataFrame, type=Float32, missingstring="NA")

```

```

Warning: `type` keyword argument is deprecated; a single type can be passed to `types` inst
@ CSV /Users/avinasharavind/.julia/packages/CSV/LiiJM/src/context.jl:356

```

	Year.AD	Flowering.DOY	Flowering.MMDD
	Float32	Float32?	Float32?
1	801.0	<i>missing</i>	<i>missing</i>
2	802.0	<i>missing</i>	<i>missing</i>
3	803.0	<i>missing</i>	<i>missing</i>
4	804.0	<i>missing</i>	<i>missing</i>
5	805.0	<i>missing</i>	<i>missing</i>
6	806.0	<i>missing</i>	<i>missing</i>
7	807.0	<i>missing</i>	<i>missing</i>
8	808.0	<i>missing</i>	<i>missing</i>
9	809.0	<i>missing</i>	<i>missing</i>
10	810.0	<i>missing</i>	<i>missing</i>
11	811.0	<i>missing</i>	<i>missing</i>
12	812.0	92.0	401.0
13	813.0	<i>missing</i>	<i>missing</i>
14	814.0	<i>missing</i>	<i>missing</i>
15	815.0	105.0	415.0
16	816.0	<i>missing</i>	<i>missing</i>
17	817.0	<i>missing</i>	<i>missing</i>
18	818.0	<i>missing</i>	<i>missing</i>
19	819.0	<i>missing</i>	<i>missing</i>
20	820.0	<i>missing</i>	<i>missing</i>
21	821.0	<i>missing</i>	<i>missing</i>
22	822.0	<i>missing</i>	<i>missing</i>
23	823.0	<i>missing</i>	<i>missing</i>
24	824.0	<i>missing</i>	<i>missing</i>
...	...	...	...

```
temp_dat = CSV.read("data/HadCRUT5.1Analysis_gl.txt", delim=" ",
  ignorerepeated=true,
  header=["Year", "January", "February", "March", "April", "May", "June", "July", "August",
  silencewarnings=true, DataFrame)
temp_dat = temp_dat[3:2:nrow(temp_dat), :]
start = findfirst==(1851.), temp_dat[!, "Year"])
finish = findfirst==(2015.), temp_dat[!, "Year"])

temp_dat = temp_dat[start:finish, :]
```

	Year	January	February	March	April	May	June	July	August	
	Int64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	1851	-0.266	-0.467	-0.641	-0.528	-0.222	-0.207	-0.136	-0.148	...
2	1852	-0.346	-0.496	-0.567	-0.575	-0.113	-0.077	0.009	-0.149	...
3	1853	-0.204	-0.399	-0.273	-0.401	-0.27	-0.124	-0.072	-0.04	...
4	1854	-0.403	-0.392	-0.247	-0.327	-0.277	-0.302	-0.162	-0.233	...
5	1855	-0.17	-0.313	-0.345	-0.211	-0.368	-0.333	-0.315	-0.202	...
6	1856	-0.083	-0.347	-0.371	-0.314	-0.293	-0.225	-0.223	-0.249	...
7	1857	-0.494	-0.442	-0.488	-0.675	-0.654	-0.291	-0.404	-0.356	...
8	1858	-0.336	-0.488	-0.566	-0.43	-0.523	-0.36	-0.341	-0.288	...
9	1859	-0.405	-0.351	-0.322	-0.189	-0.17	-0.24	-0.29	-0.266	...
10	1860	-0.39	-0.478	-0.789	-0.31	-0.371	-0.213	-0.211	-0.137	...
11	1861	-0.905	-0.681	-0.422	-0.396	-0.697	-0.272	-0.349	-0.098	...
12	1862	-0.83	-0.797	-0.538	-0.533	-0.345	-0.275	-0.356	-0.5	...
13	1863	-0.087	-0.284	-0.414	-0.336	-0.391	-0.418	-0.357	-0.335	...
14	1864	-0.825	-0.605	-0.531	-0.557	-0.346	-0.116	-0.287	-0.344	...
15	1865	-0.23	-0.513	-0.717	-0.545	-0.495	-0.251	-0.297	-0.247	...
16	1866	-0.23	-0.472	-0.748	-0.549	-0.599	-0.13	-0.053	-0.291	...
17	1867	-0.501	-0.408	-0.712	-0.489	-0.616	-0.35	-0.259	-0.325	...
18	1868	-0.707	-0.711	-0.332	-0.384	-0.282	-0.298	-0.047	-0.195	...
19	1869	-0.304	0.137	-0.651	-0.412	-0.311	-0.396	-0.432	-0.178	...
20	1870	-0.214	-0.457	-0.495	-0.357	-0.304	-0.207	-0.083	-0.256	...
21	1871	-0.591	-0.688	-0.171	-0.222	-0.348	-0.27	-0.073	-0.324	...
22	1872	-0.594	-0.627	-0.519	-0.366	-0.121	-0.267	-0.109	0.033	...
23	1873	-0.237	-0.272	-0.39	-0.462	-0.319	-0.297	-0.158	-0.217	...
24	1874	-0.182	-0.429	-0.664	-0.422	-0.311	-0.556	-0.171	-0.287	...
...	...	...	...	...	...	...	...	...	...	...

```
#MLE Log-Likelihood Estimationm
function linear_loglik(theta, blossoms, march)
    beta0, beta1, sigma = theta
    mean = beta0 .+ (beta1*march)
    ll = sum(logpdf.(skipmissing.(Normal.(mean, sigma), blossoms)))
    return ll
end

lb = [0.0, -1000.0, 1.0]
ub = [1000.0, 1000.0, 100.0]
theta0 = [500.0, 0.0, 50.0]
optim_out = Optim.optimize(theta -> -linear_loglik(theta, blossoms[:, " Flowering.DOY"], temp)
theta_mle = optim_out.minimizer
@show theta_mle
```

```
MethodError: MethodError: no method matching skipmissing(::Normal{Float64}, ::Float32)
The function `skipmissing` exists, but no method is defined for this combination of argument
```

```
Closest candidates are:
```

```
  skipmissing(::Any)
    @ Base missing.jl:232
```

```
MethodError: no method matching skipmissing(::Normal{Float64}, ::Float32)
```

```
The function `skipmissing` exists, but no method is defined for this combination of argument
```

```
Closest candidates are:
```

```
  skipmissing(::Any)
    @ Base missing.jl:232
```

```
Stacktrace:
```

```
[1] _broadcast_getindex_evalf
    @ ./broadcast.jl:678 [inlined]

[2] _broadcast_getindex
    @ ./broadcast.jl:651 [inlined]

[3] _getindex
    @ ./broadcast.jl:675 [inlined]

[4] _broadcast_getindex
    @ ./broadcast.jl:650 [inlined]

[5] getindex
```

```

@ ./broadcast.jl:610 [inlined]

[6] copy

@ ./broadcast.jl:911 [inlined]

[7] materialize

@ ./broadcast.jl:872 [inlined]

[8] linear_loglik(theta::Vector{Float64}, blossoms::Vector{Union{Missing, Float32}}, march

@ Main ~/Documents/Spring_2026/BEE4850/hw/hw04-avinasharavind/jl_notebook_cell_df34fa98e

[9] (::var"#35#36")(theta::Vector{Float64})

@ Main ~/Documents/Spring_2026/BEE4850/hw/hw04-avinasharavind/jl_notebook_cell_df34fa98e

[10] prepare_gradient_nokwarg(::Val{true}, ::Function, ::ADTypes.AutoFiniteDiff{Val{:centra

@ DifferentiationInterfaceFiniteDiffExt ~/.julia/packages/DifferentiationInterface/afUhd

[11] #prepare_gradient#68

@ ~/.julia/packages/DifferentiationInterface/afUhd/src/first_order/gradient.jl:11 [inlin

[12] prepare_gradient

@ ~/.julia/packages/DifferentiationInterface/afUhd/src/first_order/gradient.jl:8 [inlined

[13] OnceDifferentiable(f::Function, x_seed::Vector{Float64}, F::Float64, DF::Vector{Float64

@ NLSolversBase ~/.julia/packages/NLSolversBase/g0qwS/src/objective_types/onedifferentia

[14] #OnceDifferentiable#15

@ ~/.julia/packages/NLSolversBase/g0qwS/src/objective_types/onedifferentiable.jl:27 [in

[15] OnceDifferentiable (repeats 2 times)

@ ~/.julia/packages/NLSolversBase/g0qwS/src/objective_types/onedifferentiable.jl:22 [in

[16] optimize(f::Function, l::Vector{Float64}, u::Vector{Float64}, x0::Vector{Float64}, F::

```



```

@ Optim ~/.julia/packages/Optim/DtV5C/src/multivariate/solvers/constrained/fminbox.jl:280
[17] optimize (repeats 2 times)

@ ~/.julia/packages/Optim/DtV5C/src/multivariate/solvers/constrained/fminbox.jl:280 [inl
[18] optimize(f::Function, l::Vector{Float64}, u::Vector{Float64}, x0::Vector{Float64})

@ Optim ~/.julia/packages/Optim/DtV5C/src/multivariate/solvers/constrained/fminbox.jl:280
[19] top-level scope

@ ~/Documents/Spring_2026/BEE4850/hw/hw04-avinasharavind/jl_notebook_cell_df34fa98e69747

```

Problem 1.2

Calculate the log-loss (negative log-probability) and mean-squared error metrics for the fitted model on the in-sample data.

Problem 1.3

Conduct a 5-fold cross-validation for the model using both of these metrics. How do they compare to the in-sample metrics?

Problem 1.4

What conclusions can you draw about the model's generalizability?

Problem 2

Let's re-examine the influence of air pollution and temperature on mortality in Chicago, IL. The `data/chicago.csv` dataset (originally from the [gamaair R package](#)) contains data on the relationship between environmental conditions (temperature and air quality) and deaths in Chicago from 1987–2000. The variables are:

- the total number of (non-accidental) deaths each day (`death`);
- the median density over the city of large pollutant particles (PM10; `pm10median`);
- the median density of smaller pollutant particles (PM2.5; `pm25median`);
- the median concentration of ozone (O₃) in the air (`o3median`); the median concentration of sulfur dioxide (SO₂) in the air (`so2median`);
- the time in days (`time`); and the daily mean temperature (`tmpd`).

The pollution variables have been standardized to represent deviations from the overall median and the temperature is in Fahrenheit. The days are given as a number of days relative to December 31, 1993. Note that some values are missing; these can be treated as missing at random and dropped in your regressions.

```
chicago_dat = CSV.read("data/chicago.csv", DataFrame; delim=',', header=true, missingstring=
day_zero = Date("1993-12-31")
chicago_dat.Date = day_zero .+ Day.(chicago_dat.time .+ 0.5)

plot(chicago_dat.Date, chicago_dat.death, lw=2, xlabel="Date", ylabel="Non-Accidental Deaths")
```

Problem 2.1

Load and plot the deaths data. Linearly regress the number of deaths against time and report the coefficients for the fitted model. Add the regression line to your plot. What does this regression tell you?

Problem 2.2

Now linearly regress the deaths data against temperature. Once again, report the coefficients, and add the regression line to your plot. What does this regression tell you about the influence of temperature?

Problem 2.3

Expand your previous model by regressing deaths against temperature and PM2.5 density. Report the coefficients and add this regression line to your plot.

Problem 2.4

Conduct a 5-fold cross-validation for the temperature only and temperature and PM2.5 models, using both the mean-squared-error and log-probability metrics. Are there any structures in the dataset that you need to account for when developing your cross-validation folds? Which model performs the best? What can you conclude from this analysis?

Problem 2.5

Compute the AICs for the three fitted models and compare them. What do they tell you about the relative evidence for the influence of temperature and PM2.5 density on deaths? How do your conclusions based on AIC compare to those you might draw from your regression coefficients, plots, and cross-validation?

References